ԼԱԲՈՐԱՏՈՐ ԱՇԽԱՏԱՆՔ 6

Դասերի օգտագործմամբ բարդ խնդիրներ (C++) Պահանջ. յուրաքանչյուր խնդրի համար սահմանել class՝ private դաշտերով, public կոնստրուկտորով և մեթոդներով: Օգտագործել օբյեկտների զանգված կամ vector, ապահովել մուտքային տվյալների ստուգում: Հիմնական տրամաբանությունը իրականացնել դասի մեթոդներով և անհրաժեշտ օժանդակ ֆունկցիաներով: Պատրաստի STL դասավորման գործառույթը կարելի է կիրառել միայն այն դեպքում, երբ դասախոսը թույլատրում է:

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

class Course {
private:
    string code;
    string title;
    int credits;
    int capacity;
    int enrolled;
    vector<string> prerequisites;

public:
    Course(string c, string t, int cr, int cap, vector<string> prereq = {}) {
        code = c;
        title = t;
        credits = (cr > 0) ? cr : 1;
        capacity = (cap > 0) ? cap : 1;
        enrolled = 0;
        prerequisites = prereq;
    }

    string getCode() const { return code; }
    string getTitle() const { return title; }
    int getCapacity() const { return capacity; }
    int getEnrolled() const { return enrolled; }
```

```
double getOccupancyRate() const {
    return (double)enrolled / capacity * 100.0;
}

bool canEnroll(const vector<string>& completedCourses) const {
    if (enrolled >= capacity) return false;
    for (const auto& prereq : prerequisites) {
        if (find(completedCourses.begin(), completedCourses.end(), prereq) == co
            return false; // Նախապայմանը բավարարված չէ
        }
    }
    return true;
}

bool enroll() {
    if (enrolled < capacity) {
        enrolled++;
        return true;
    }
    return false;
}

bool cancelEnrollment() {
    if (enrolled > 0) {
        enrolled--;
        return true;
    }
    return false;
}

void print() const {
    cout << "[" << code << "] " << title << " | Կրեդիտներ: " << credits
         << " | Տեղեր: " << enrolled << "/" << capacity << " (" << getOccupancyR
```

```
        cout << "[" << code << "] " << title << " | Կրեդիտներ: " << credits
             << " | Տեղեր: " << enrolled << "/" << capacity << " (" << getOccupancyR
    }
};

int main() {
    // Ստեղծում ենք 10 դասընթաց՝ իրենց նախապայմաններով
    vector<Course> courses = {
        Course("CS101", "Ծրագրավորման հիմունքներ", 5, 30),
        Course("CS102", "Օբյեկտային Ծրագրավորում (OOP)", 6, 25, {"CS101"}),
        Course("MATH101", "Բարձրագույն մաթեմատիկա", 5, 40),
        Course("MATH102", "Գծային հանրահաշիվ", 4, 30, {"MATH101"}),
        Course("CS201", "Տվյալների կառույցներ և ալգորիթմներ", 6, 20, {"CS102", "MATH
        Course("DB101", "Տվյալների բազաներ", 5, 25, {"CS101"}),
        Course("WEB101", "Վեբ ծրագրավորման հիմունքներ", 4, 35, {"CS101"}),
        Course("AI101", "Արհեստական բանականության հիմունքներ", 6, 15, {"CS201", "MAT
        Course("NET101", "Համակարգչային ցանցեր", 4, 25, {"CS101"}),
        Course("SEC101", "Տեղեկատվական անվտանգություն", 5, 20, {"NET101"})
    };

    // Ուսանողի անցած առարկաների ցանկը
    vector<string> studentCompleted = {"CS101", "MATH101"};

    cout << "--- ԴԱՍԸՆԹԱՑՆԵՐԻ ՍԿԶԲՆԱԿԱՆ ՑԱՆԿ ---\n";
    for (auto& c : courses) {
        c.print();
        // Փորձենք գրանցել ուսանողին, եթե հնարավոր է
        if (c.canEnroll(studentCompleted)) {
            c.enroll();
        }
```

```
        if (c.canEnroll(studentCompleted)) {
            c.enroll();
        }
    }

    cout << "\n--- ԳՐԱՆՑՈՒՄՆԵՐԻՑ ՀԵՏՈ ---\n";
    for (auto& c : courses) {
        c.print();
    }

    // Գտնել ամենաբարձր զբաղվածության տոկոսով դասընթացը
    double maxRate = -1;
    string topCourse = "";
    for (const auto& c : courses) {
        if (c.getOccupancyRate() > maxRate) {
            maxRate = c.getOccupancyRate();
            topCourse = c.getTitle();
        }
    }
    cout << "\nԱմենաբարձր զբաղվածությամբ դասընթացը: " << topCourse << " (" << maxRat

    return 0;
}
```